COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	10 / 01 / 2026	**DATE SUBMITTED:**	
NAME:	Elijah I. Dizor	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 10.1 Graphs

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

CODE

main

```
#include <iostream>

using namespace std;

//adjacency list
struct adjNode {
    int val, cost;
    adjNode* next;
};

//store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph {
private:

    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;

        newNode->next = head;
        return newNode;
    }

public:
    int N;
    adjNode** head;
```

```
    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
        head = new adjNode*[N]();
        this->N = N;

        for (int i = 0; i < N; ++i) {
            head[i] = nullptr;
        }

        for (unsigned i = 0; i < n; i++) {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;

            adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);

            head[start_ver] = newNode;
        }
    }

    // Destructor
    ~DiaGraph() {
        for (int i = 0; i < N; i++) {
            adjNode* curr = head[i];
            while (curr != nullptr) {
                adjNode* tmp = curr;
                curr = curr->next;
                delete tmp;
            }
        }
        delete[] head;
    }
};

void display_AdjList(adjNode* ptr, int i)
{
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val
                  << ", " << ptr->cost << ") ";
        ptr = ptr->next;
    }
    std::cout << std::endl;
}
```

```
int main()
{
    // graph edges array.
    graphEdge edges[] = {
        {0,1,2}, {0,2,4}, {1,4,3}, {2,3,2}, {3,1,4}, {4,3,3}
    };

    int N = 6; //number of vertices in the graph

    //number of edges
    int n = sizeof(edges) / sizeof(edges[0]);

    // cons graph
    DiaGraph diagraph(edges, n, N);


    std::cout << "    Graph adjacency list " << std::endl
              << "start_vertex, end_vertex, weight: \n" << std::endl;

    for (int i = 0; i < N; i++)
    {
        // display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
    }

    return 0;
}
```

Header

```cpp
#ifndef DIAGRAPH_H
#define DIAGRAPH_H

#include <iostream>

// Adjacency list node structure
struct adjNode {
    int val;
    int cost;
    adjNode* next;
};

struct graphEdge {
    int start_ver;
    int end_ver;
    int weight;
};

class DiaGraph {
private:
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head;
        return newNode;
    }

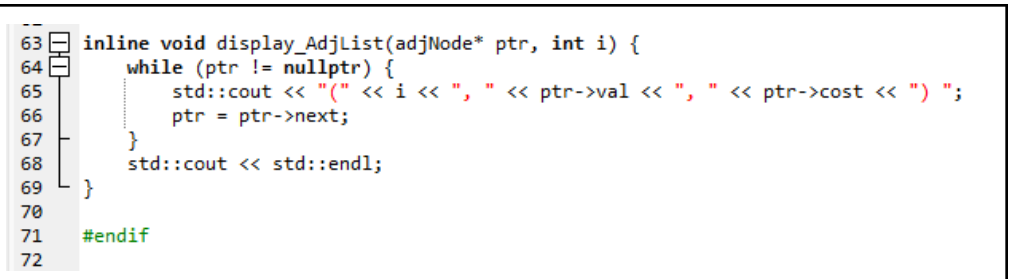
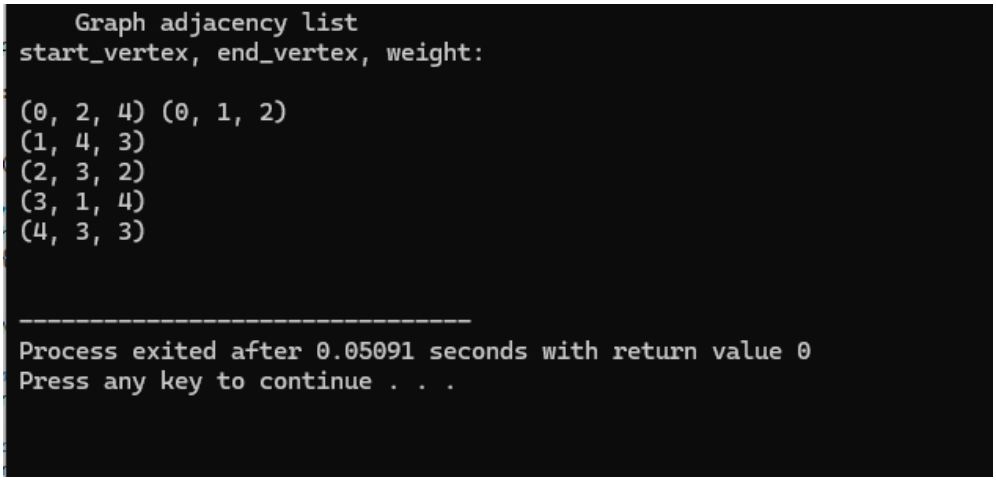
public:
    int N;              // number of vertices
    adjNode** head;

    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
        this->N = N;
        head = new adjNode*[N]();

        // Construct directed graph by adding edges
        for (int i = 0; i < n; i++) {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;

            adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);
            head[start_ver] = newNode;
        }
    }

    // Destructor to clean up dynamically allocated memory
    ~DiaGraph() {
        for (int i = 0; i < N; i++) {
            adjNode* curr = head[i];
            while (curr != nullptr) {
                adjNode* tmp = curr;
                curr = curr->next;
                delete tmp;
            }
        }
        delete[] head;
    }
};
```

	
OUTPUT	

```
63  inline void display_AdjList(adjNode* ptr, int i) {
64      while (ptr != nullptr) {
65          std::cout << "(" << i << ", " << ptr->val << ", " << ptr->cost << ") ";
66          ptr = ptr->next;
67      }
68      std::cout << std::endl;
69  }
70
71  #endif
72
```

```
    Graph adjacency list
start_vertex, end_vertex, weight:

(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)


--------------------------------
Process exited after 0.05091 seconds with return value 0
Press any key to continue . . .
```

It stores the edges using an array of graphedge, then converted into dynamic linked lists. The diagraph allocates an array of pointers which is the head, where each index represents a vertex. It then loops through the edges and inserts each destination node at the front of that vertex linked list

ILO B: Create C++ code for implementing graph traversal algorithms such as Breadth-First and Depth-First Search

B.1 Depth-First Search

CODE	Main

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;
    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};
```

```cpp
template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N)
    {
    }

    size_t vertices() const
    {
        return V;
    }

    const std::vector<Edge<T>>& edges() const
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    std::vector<Edge<T>> outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (const auto &e : edge_list)
        {
            if (e.src == v)
                edges_from_v.emplace_back(e);
        }
        return edges_from_v;
    }
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
```

```cpp
private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (size_t i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        std::vector<Edge<T>> edges = G.outgoing_edges(i);
        for (const auto &e : edges)
        {
            os << "{" << e.dest << ": " << e.weight << "}, ";
        }
        os << std::endl;
    }
    return os;
}
```

```cpp
template <typename T>
std::vector<size_t> depth_first_search(const Graph<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1);
    while (!stack.empty())
    {
        size_t current_vertex = stack.top();
        stack.pop();
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            std::vector<Edge<T>> edges = G.outgoing_edges(current_vertex);
            for (const auto &e : edges)
            {
                if (visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}
```

```cpp
template <typename T>
Graph<T> create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;

    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

    for (const auto &i : edges)
    {
        for (const auto &j : i.second)
        {
            G.add_edge(Edge<T>{i.first, j.first, j.second});
        }
    }
    return G;
}

template <typename T>
void test_DFS()
{
    Graph<T> G = create_reference_graph<T>();
    std::cout << G << std::endl;
    std::cout << "DFS Order of vertices: " << std::endl;
    std::vector<size_t> dfs_visit_order = depth_first_search(G, 1);
    for (size_t v : dfs_visit_order)
        std::cout << v << std::endl;
}

int main()
{
    using T = unsigned;
    test_DFS<T>();
    return 0;
}
```

Header

```cpp
#ifndef GRAPH_H
#define GRAPH_H

#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;
    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};
```

```cpp
template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N)
    {
    }
    auto vertices() const
    {
        return V;
    }
    const auto &edges() const
    {
        return edge_list;
    }
    void add_edge(Edge<T> &&e)
    {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }
    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
                edges_from_v.emplace_back(e);
        }
        return edges_from_v;
    }
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};
```

```cpp
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
        {
            os << "{" << e.dest << ": " << e.weight << "}, ";
        }
        os << std::endl;
    }
    return os;
}

template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1);
    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto &e : G.outgoing_edges(current_vertex))
            {
                if (visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}

```

```cpp
template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};
    for (auto &i : edges)
    {
        for (auto &j : i.second)
        {
            G.add_edge(Edge<T>{i.first, j.first, j.second});
        }
    }
    return G;
}

template <typename T>
void test_DFS()
{
    auto G = create_reference_graph<T>();
    std::cout << G << std::endl;
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
        std::cout << v << std::endl;
}

#endif
```

OUTPUT

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

The struct edge represents an edge, then it tracks the source, destination vertex, and weight of type T. It overloads operators so edges can be compared by their weights. Class graph implements an edge list structure, it places every single edge created across the graph into flat dynamic vector arrays.

B.2. Breadth-First Search

CODE

```
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>
#include <queue>

template <typename T> class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const { return this->weight < e.weight; }
    inline bool operator>(const Edge<T> &e) const { return this->weight > e.weight; }
};

```

```cpp
template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N) {}

    size_t vertices() const { return V; }

    const std::vector<Edge<T>>& edges() const { return edge_list; }

    void add_edge(Edge<T> &&e)
    {
        if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    // CHANGED: changed auto to explicit vector return type
    std::vector<Edge<T>> outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v) edges_from_v.emplace_back(e);
        }
        return edges_from_v;
    }

    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i <= G.vertices(); i++)
    {
        os << i << ":\t";
        std::vector<Edge<T>> edges = G.outgoing_edges(i);
        for (auto &e : edges)
        {
            os << "{" << e.dest << ": " << e.weight << "}, ";
        }
        os << std::endl;
    }
    return os;
}

template <typename T>
std::vector<size_t> depth_first_search(const Graph<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    stack.push(1);

    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();

```

```
89          if (visited.find(current_vertex) == visited.end())
90          {
91              visited.insert(current_vertex);
92              visit_order.push_back(current_vertex);
93
94              for (auto e : G.outgoing_edges(current_vertex))
95              {
96                  if (visited.find(e.dest) == visited.end())
97                  {
98                      stack.push(e.dest);
99                  }
100             }
101         }
102     }
103     return visit_order;
104 }
105
106
107 template <typename T>
108 std::vector<size_t> breadth_first_search(const Graph<T> &G, size_t start_vertex)
109 {
110     std::queue<size_t> queue;
111     std::vector<size_t> visit_order;
112     std::set<size_t> visited;
113
114     queue.push(start_vertex);
115
116     while (!queue.empty())
117     {
118         auto current_vertex = queue.front();
119         queue.pop();
120
121         if (visited.find(current_vertex) == visited.end())
122         {
123             visited.insert(current_vertex);
```

```
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex))
            {
                if (visited.find(e.dest) == visited.end())
                {
                    queue.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}

template <typename T>
Graph<T> create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;

    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

    for (auto &i : edges)
    {
        for (auto &j : i.second)
        {
            G.add_edge(Edge<T>{i.first, j.first, j.second});
        }
    }
    return G;
}

template <typename T>
void test_BFS()
{
    Graph<unsigned> G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    std::cout << "BFS Order of vertices: " << std::endl;
    std::vector<size_t> bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
}

int main()
{
    using T = unsigned;
    test_BFS<T>();
    return 0;
}
```

OUTPUT

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},
9:

BFS Order of vertices:
1
2
5
4
8
3
6
7
```

Based on the code, it represents a one way path from a source vertex, to a destination vertex. The 'add_edge' takes a new edge and verifies if its source and destination values are valid. If they are valid, it pushes the edge into the flat dynamic array edge_list. Because a flat edge list is hard to search quickly, and the 'outgoing_edges' function scans through the entire edge list to find and return every edge where the source is.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

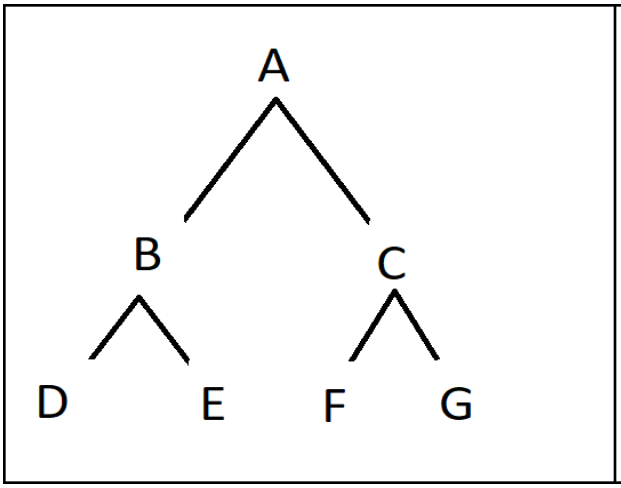
Copy and paste the questions from the lab and include your answers after.

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.

- Depth-FIrst Search (DFS). because it explores as far as possible along each branch or path until it reaches a dead end of the vertex. The algorithm backtracks using a stack to the most recent vertex with unexplored neighbors. After backtracking, it then continues to explore other paths stemming from that same vertex.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.



Tree traversals

Pre order: A > B > D > E > C > F > G

In order: D > B > E > A > F > C > G

Post order: D > E > B > F > G > C > A

It is similar to the three binary tree traversals such as pre order, in order, and post order, but there is a difference. Both graphs and trees use a stack model to explore modes, both will go straight down a single path as deep as possible until it hits a leaf node with no other pathways left. Both pop items off the stack to back up to the most recent junction points.

The difference is that graphs such as DFS can have loops or multiple ways to reach the same location, and they require an extra safety net that trees do not need.

-

3. In the performed code, what data structure is used to implement the Breadth First Search?

A queue follows the first in, and first out principle, to ensure that nodes are processed in the exact order they are discovered. By popping the oldest nodes from the front of the queue, and adding its unvisited nodes to the back, the algorithm systematically visits all nodes at the current distance level before moving deeper into the graph

4. How many times can a node be visited in the BFS?

Each node can be visited exactly one time. Because as soon as the node is taken out of the queue and processed, it is now added to a tracking set to be visited. If a node has already been visited, the algorithm ignores it and prevents it from being processed a second time.

Conclusion / Learning Analysis

- Graphs are standard for modeling complex and relational data where entities are connected. When creating graphs and choosing how to structure the internal storage dictates performance. So far I learned that binary search tree traversals and both BFS and DFS are similar but have their differences in terms of performance and certain conditions must be met, to acquire and implement those methods. Based on what I did on this activity, I can say that I managed to complete it and have an understanding on how to implement these graphs. As for my improvements, I would find better ways to solve complex real life scenarios by the use of graphs and tree traversals, to gather data and increase their efficiency.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.